



Part 1: Heart Disease Prediction

```
In [1]: import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
In [2]: # read data provided on Moodle
df = pd.read_csv("train_val.csv", sep=",")
```

```
In [3]: df_test = pd.read_csv("test.csv", sep=",")
```

Q1: Exploratory Data Analysis (3 Pts)

Get familiar with the dataset by exploring the different features, their distribution, and the labels (1 Pt).

Features:

- Age: age of the patient [years]
- Sex: sex of the patient [M: Male, F: Female]
- ChestPainType: chest pain type [TA: Typical Angina, ATA: Atypical Angina, NAP: Non-Anginal Pain, ASY: Asymptomatic]
- RestingBP: resting blood pressure [mm Hg]
- Cholesterol: serum cholesterol [mm/dl]
- FastingBS: fasting blood sugar [1: if FastingBS > 120 mg/dl, 0: otherwise]
- RestingECG: resting electrocardiogram results [Normal: Normal, ST: having ST-T wave abnormality (T wave inversions and/or ST elevation or depression of > 0.05 mV), LVH: showing probable or definite left ventricular hypertrophy by Estes' criteria]
- MaxHR: maximum heart rate achieved [Numeric value between 60 and 202]
- ExerciseAngina: exercise-induced angina [Y: Yes, N: No]
- Oldpeak: oldpeak = ST [Numeric value measured in depression]
- ST_Slope: the slope of the peak exercise ST segment [Up: upsloping, Flat: flat, Down: downsloping]

Class label (target):

- HeartDisease: output class [1: heart disease, 0: Normal]

(taken from Kaggle)

```
In [4]: # features and label
df.columns
```

```
Out[4]: Index(['Age', 'Sex', 'ChestPainType', 'RestingBP', 'Cholesterol', 'FastingBS',
              'RestingECG', 'MaxHR', 'ExerciseAngina', 'Oldpeak', 'ST_Slope',
              'HeartDisease'],
              dtype='object')
```

```
In [5]: df.head()
```

```
Out[5]:
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR
0	40	M	ATA	140	289	0	Normal	
1	49	F	NAP	160	180	0	Normal	
2	37	M	ATA	130	283	0	ST	
3	48	F	ASY	138	214	0	Normal	
4	54	M	NAP	150	195	0	Normal	

```
In [6]: # There are no obvious NAs
# we have int, float and object columns
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Age                   918 non-null   int64
1   Sex                   918 non-null   object
2   ChestPainType         918 non-null   object
3   RestingBP             918 non-null   int64
4   Cholesterol           918 non-null   int64
5   FastingBS             918 non-null   int64
6   RestingECG           918 non-null   object
7   MaxHR                 918 non-null   int64
8   ExerciseAngina        918 non-null   object
9   Oldpeak               918 non-null   float64
10  ST_Slope              918 non-null   object
11  HeartDisease          918 non-null   int64
dtypes: float64(1), int64(6), object(5)
memory usage: 86.2+ KB
```

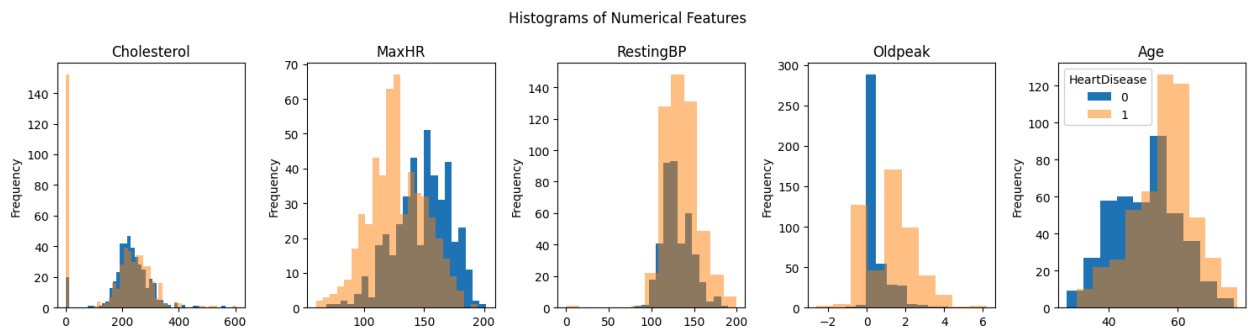
plot feature distributions

```
In [7]: # identify numeric and categorical columns
feat_cat = {'Sex', 'ChestPainType', 'FastingBS', 'RestingECG', 'ST_Slope', 'ExerciseAngina', 'HeartDisease'}
feat_num = {'Age', 'RestingBP', 'Cholesterol', 'MaxHR', 'Oldpeak'}
```

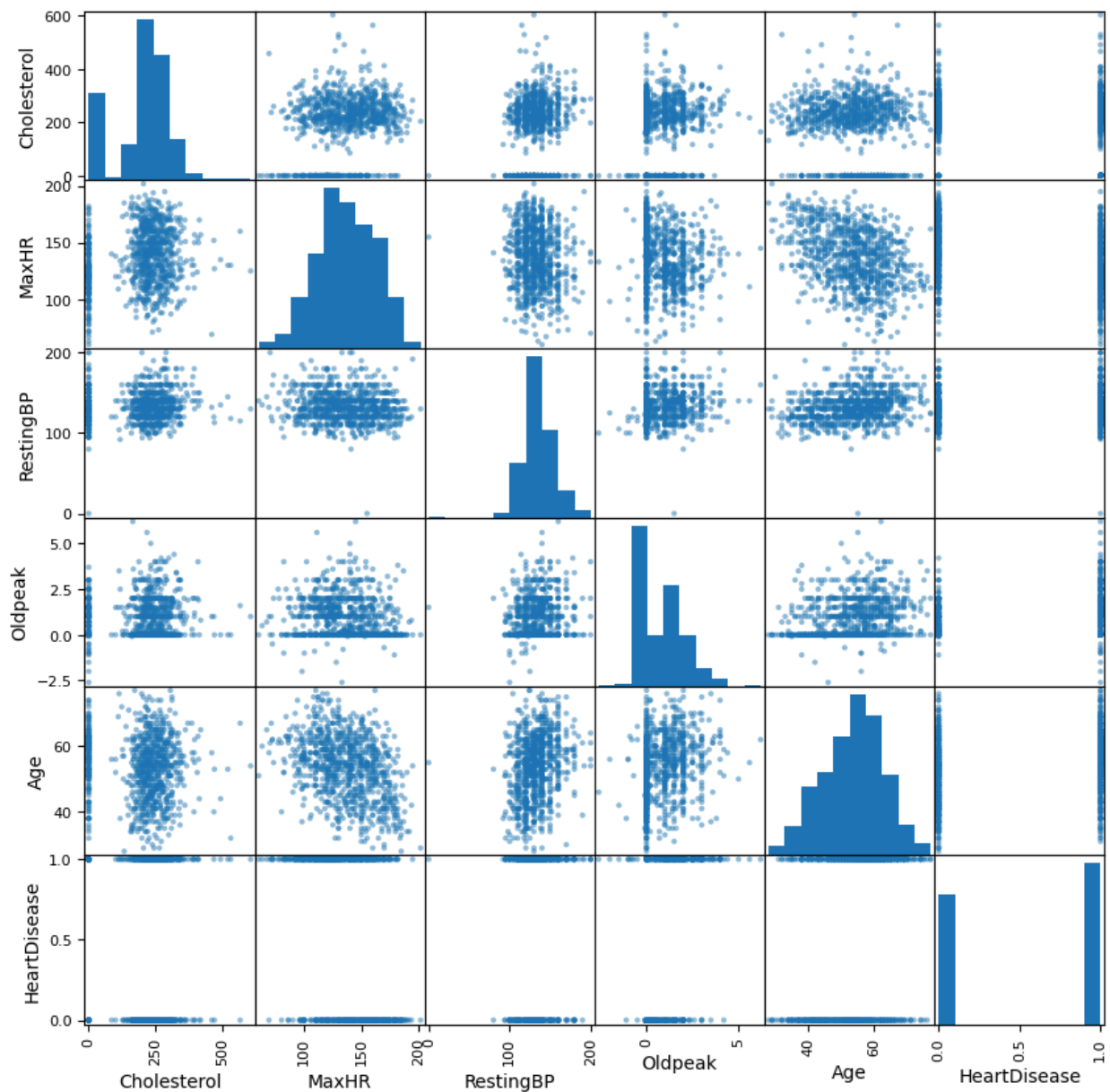
```
label = 'HeartDisease'
assert len(feats_cat | feats_num) == len(df.columns) - 1
```

```
In [8]: # numeric features
fig, axs = plt.subplots(1, len(feats_num), figsize=(15,4))
fig.suptitle('Histograms of Numerical Features')

for i, col in enumerate(feats_num):
    ax=axs[i]
    df.loc[df[label]==0,col].plot(kind="hist", title=col, bins=math.floor(len(
    df.loc[df[label]==1,col].plot(kind="hist", title=col, bins=math.floor(len(
plt.legend(title=label)
fig.tight_layout()
plt.show()
```



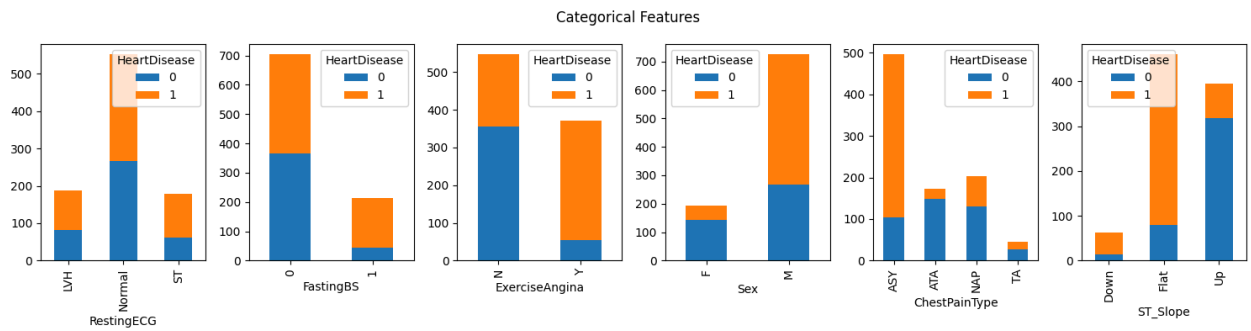
```
In [9]: from pandas.plotting import scatter_matrix
_ = scatter_matrix(df[list(feats_num) + [label]], figsize=(10,10))
```



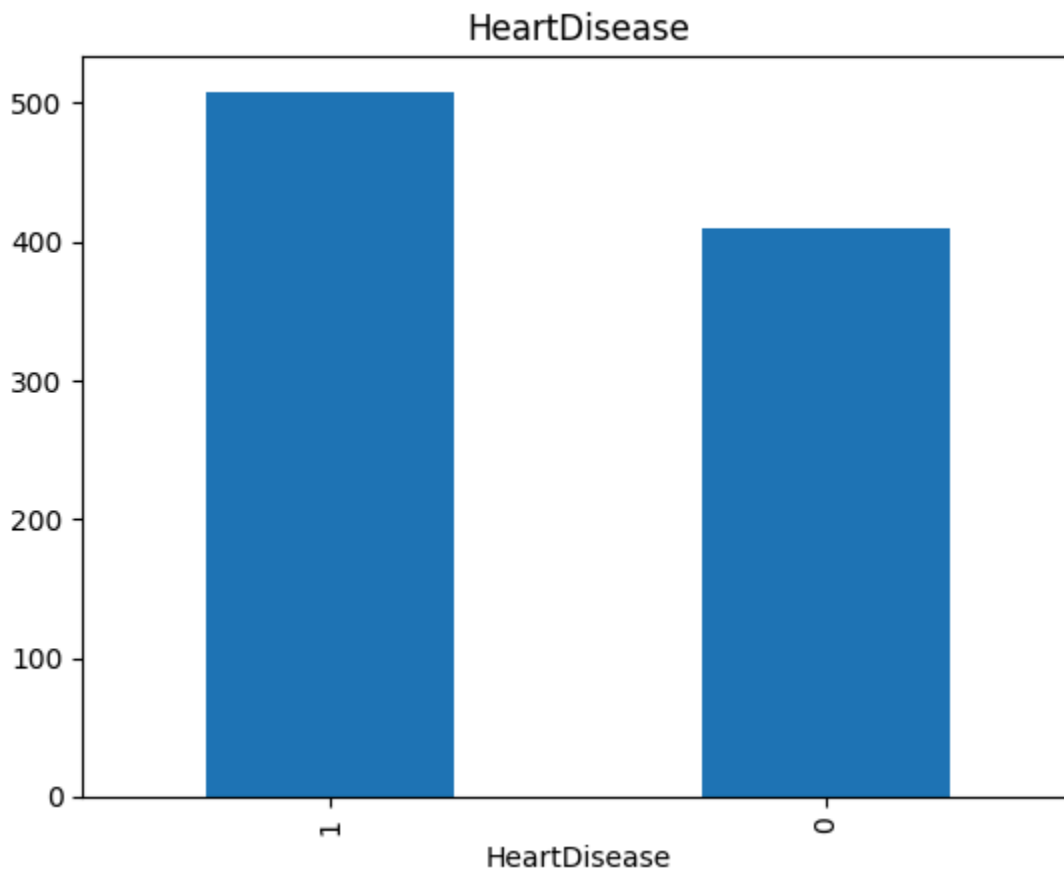
```
In [10]: # categorical features

fig, axs = plt.subplots(1, len(feats_cat), figsize=(15,4))
fig.suptitle('Categorical Features')

for i, col in enumerate(feats_cat):
    ax = axs[i]
    df.groupby([col, label])[col].count().unstack().plot(kind="bar", stacked=True)
    # df.loc[df[label]==0, col].value_counts().plot.bar(title=col, label=0, ax=ax)
    # df.loc[df[label]==1, col].value_counts().plot.bar(title=col, label=1, ax=ax)
fig.tight_layout()
plt.show()
```



```
In [11]: # label looks rather balanced
df[label].value_counts().plot.bar(title=label)
plt.show()
```



Check for common pitfalls like missing or nonsensical data, unusual feature distribution, outliers, or class imbalance, and describe how to handle them (1 Pt).

Missing data

- no NAs

Nonsensical Data

- none

Unusual feature distributions

- Cholesterol == 0 far apart from usual distribution => looks like missing, add column indicating the missing
- RestingBP == 0 makes no sense, but only one occurrence => do nothing

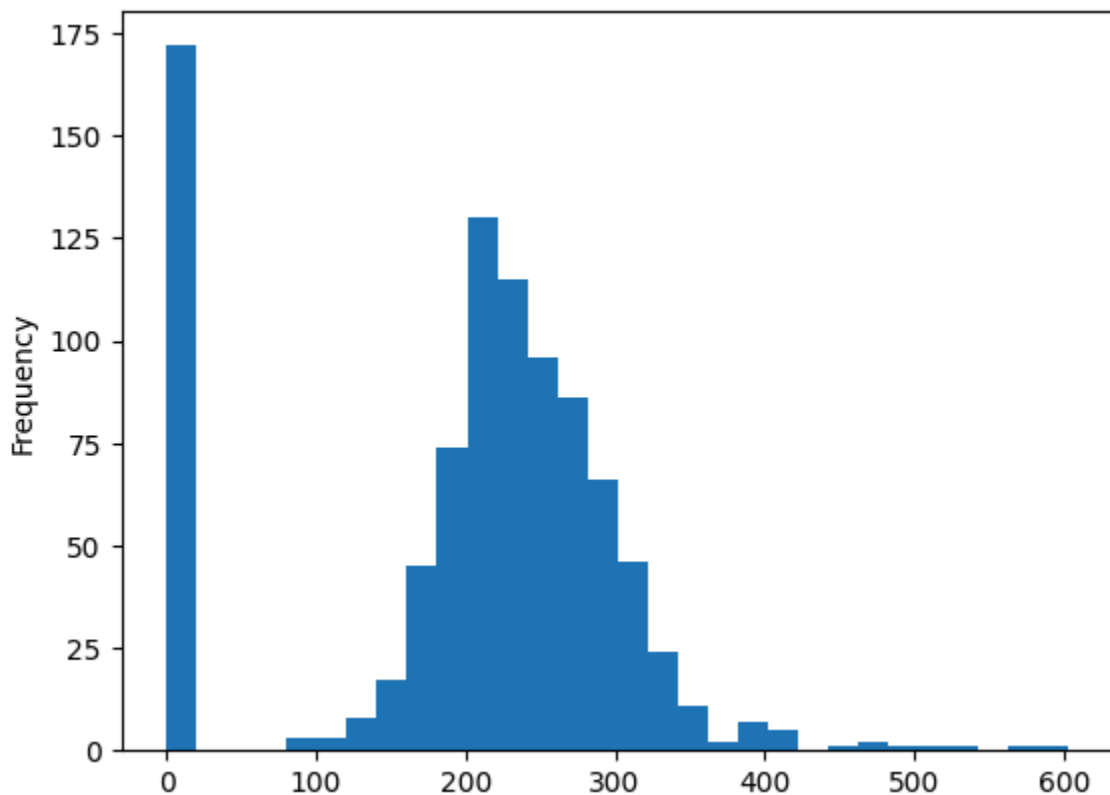
Balance

- categorical features suffer from imbalancedness; however
- label is almost balanced

```
In [12]: # Cholesterol
print((df["Cholesterol"]==0).sum(), "zeros")

df["Cholesterol"].plot(kind="hist", bins=30)
plt.show()
```

172 zeros

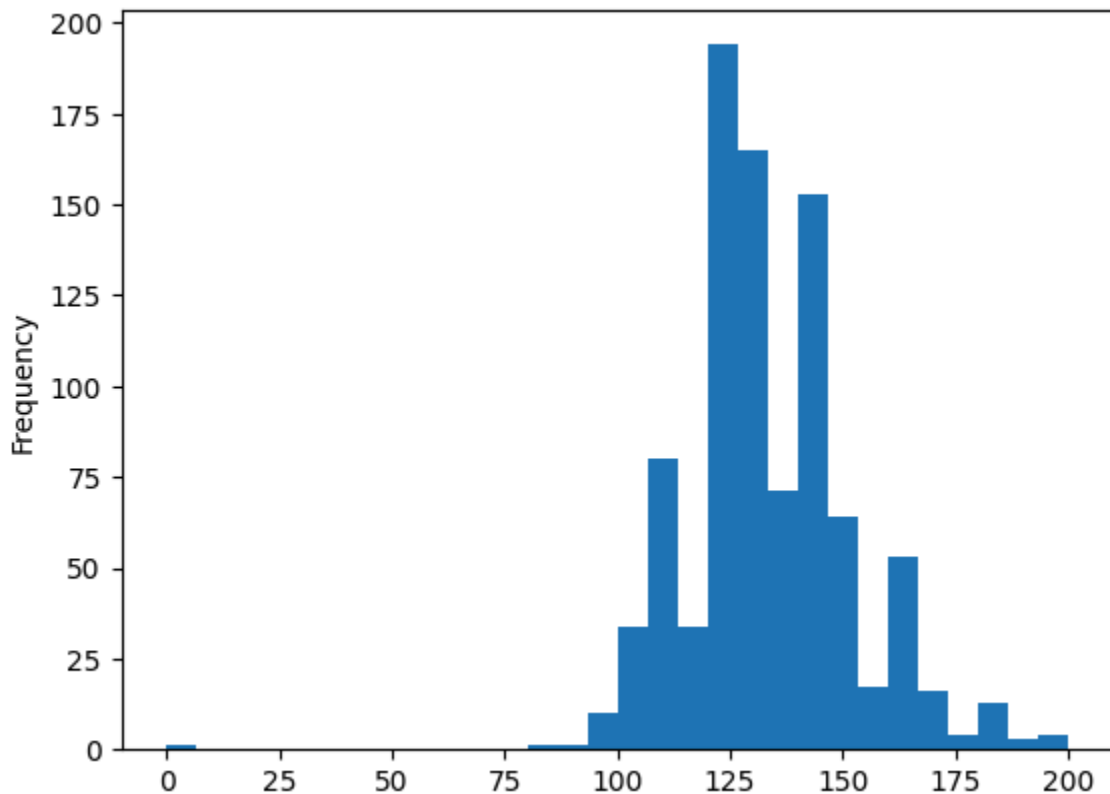


```
In [13]: # RestingBP
print((df["RestingBP"]==0).sum(), "zeros")

df["RestingBP"].plot(kind="hist", bins=30)
```

```
plt.show()
```

1 zeros



```
In [14]: df.loc[df["RestingBP"]==0]
```

```
Out[14]:
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG
449	55	M	NAP	0	0	0	Normal

```
In [15]: df.head()
```

```
Out[15]:
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	Mi
0	40	M	ATA	140	289	0	Normal	
1	49	F	NAP	160	180	0	Normal	
2	37	M	ATA	130	283	0	ST	
3	48	F	ASY	138	214	0	Normal	
4	54	M	NAP	150	195	0	Normal	

Explain how you preprocess the dataset for the remaining tasks of part 1 (1 Pt).

- split dataset into features and label
- dummy encoding of categorical features: linear model so drop first

column

```
In [16]: # add indication for missingness
df["Cholesterol_missing"] = 0
df.loc[df["Cholesterol"]==0, "Cholesterol_missing"] = 1
```

```
In [45]: df_test=pd.read_csv("dataset.csv",sep=",")
df_test.head()
```

```
Out[45]:
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR
0	63	1	3	145	233	1	0	
1	37	1	2	130	250	0	1	
2	41	0	1	130	204	0	0	
3	56	1	1	120	236	0	1	
4	57	0	0	120	354	0	1	

```
In [46]: df_test["Cholesterol_missing"] = 0
df_test.loc[df_test["Cholesterol"]==0, "Cholesterol_missing"] = 1
```

```
In [47]: df_test.columns
```

```
Out[47]: Index(['Age', 'Sex', 'ChestPainType', 'RestingBP', 'Cholesterol', 'FastingBS',
               'RestingECG', 'MaxHR', 'ExerciseAngina', 'Oldpeak', 'ST_Slope',
               'HeartDisease', 'Cholesterol_missing'],
              dtype='object')
```

```
In [48]: # add Cholesterol_missing to categorical features
feat_cat = feat_cat | {"Cholesterol_missing"}
```

```
In [49]: X = df.drop(columns="HeartDisease")
y = df["HeartDisease"]
X_test = df_test.drop(columns="HeartDisease")
y_test = df_test["HeartDisease"]
```

```
In [50]: # fill Cholesterol with median of train set
med = X["Cholesterol"].median()
X.loc[X["Cholesterol"]==0, "Cholesterol"] = med
X_test.loc[X_test["Cholesterol"]==0, "Cholesterol"] = med

# fill RestingBP with median of train set
med = df["RestingBP"].median()
X.loc[X["RestingBP"]==0, "RestingBP"] = med
X_test.loc[X_test["RestingBP"]==0, "RestingBP"] = med
```

```
In [51]: X.columns
```



```
Out[51]: Index(['Age', 'Sex', 'ChestPainType', 'RestingBP', 'Cholesterol', 'FastingBS',
               'RestingECG', 'MaxHR', 'ExerciseAngina', 'Oldpeak', 'ST_Slope',
               'Cholesterol_missing'],
              dtype='object')
```

```
In [52]: X_test.columns
```

```
Out[52]: Index(['Age', 'Sex', 'ChestPainType', 'RestingBP', 'Cholesterol', 'FastingBS',
               'RestingECG', 'MaxHR', 'ExerciseAngina', 'Oldpeak', 'ST_Slope',
               'Cholesterol_missing'],
              dtype='object')
```

```
In [53]: # 1. Encode the training features
X = pd.get_dummies(X, columns=list(feet_cat), drop_first=False)

# 2. Encode the test features
X_test = pd.get_dummies(X_test, columns=list(feet_cat), drop_first=False)

# 3. ALIGNMENT: Ensure X_test has the exact same columns as X
# This adds missing columns as 0s and removes any columns not seen during training
X_test = X_test.reindex(columns=X.columns, fill_value=0)
```

```
In [54]: assert len(set(X.columns) - set(X_test.columns)) == 0
```

```
In [55]: len(set(X_test.columns))
```

```
Out[55]: 23
```

```
In [56]: X.columns
```

```
Out[56]: Index(['Age', 'RestingBP', 'Cholesterol', 'MaxHR', 'Oldpeak', 'RestingECG_LVH',
               'RestingECG_Normal', 'RestingECG_ST', 'Cholesterol_missing_0',
               'Cholesterol_missing_1', 'FastingBS_0', 'FastingBS_1',
               'ExerciseAngina_N', 'ExerciseAngina_Y', 'Sex_F', 'Sex_M',
               'ChestPainType_ASY', 'ChestPainType_ATA', 'ChestPainType_NAP',
               'ChestPainType_TA', 'ST_Slope_Down', 'ST_Slope_Flat', 'ST_Slope_Up'],
              dtype='object')
```

```
In [57]: X_test.columns
```

```
Out[57]: Index(['Age', 'RestingBP', 'Cholesterol', 'MaxHR', 'Oldpeak', 'RestingECG_LVH',
               'RestingECG_Normal', 'RestingECG_ST', 'Cholesterol_missing_0',
               'Cholesterol_missing_1', 'FastingBS_0', 'FastingBS_1',
               'ExerciseAngina_N', 'ExerciseAngina_Y', 'Sex_F', 'Sex_M',
               'ChestPainType_ASY', 'ChestPainType_ATA', 'ChestPainType_NAP',
               'ChestPainType_TA', 'ST_Slope_Down', 'ST_Slope_Flat', 'ST_Slope_Up'],
              dtype='object')
```

Q2: Logistic Lasso Regression (5 Pts)

Describe which preprocessing steps are crucial (1 Pt) to ensure comparability of feature coefficients.

standardization

The lasso method requires initial standardization of the regressors, so that the penalization scheme is fair to all regressors. For categorical regressors, one codes the regressor with dummy variables and then standardizes the dummy variables.

```
In [58]: # standardization
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X)
X = scaler.transform(X)
X_test = scaler.transform(X_test)

feature_names = scaler.feature_names_in_
```

fit a Lasso regression model with l1 regularization (1 Pt) on the dataset

```
In [59]: from sklearn.linear_model import LogisticRegression
log = LogisticRegression(penalty='l1', solver='liblinear')
log.fit(X, y)
```

Out[59]:

LogisticRegression		
Parameters		
	penalty	'l1'
	dual	False
	tol	0.0001
	C	1.0
	fit_intercept	True
	intercept_scaling	1
	class_weight	None
	random_state	None
	solver	'liblinear'
	max_iter	100
	multi_class	'deprecated'
	verbose	0
	warm_start	False
	n_jobs	None
	l1_ratio	None

Provide performance metrics such as f1-score or balanced accuracy (1 Pt)

```
In [60]: # make predictions on test set
y_test_pred = log.predict(X_test)
```

```
In [62]: # F1 score: harmonic mean of the precision and recall.
from sklearn.metrics import f1_score
f1_score(y_true=y_test, y_pred=y_test_pred, average='macro')
```

Out[62]: 0.14705882352941177

```
In [63]: # Balanced accuracy score: average of recall obtained on each class.
from sklearn.metrics import balanced_accuracy_score
balanced_accuracy_score(y_true=y_test, y_pred=y_test_pred)
```

Out[63]: 0.2

Visualize the importance of the different features and argue how they contribute to the model's output. (1 Pt)

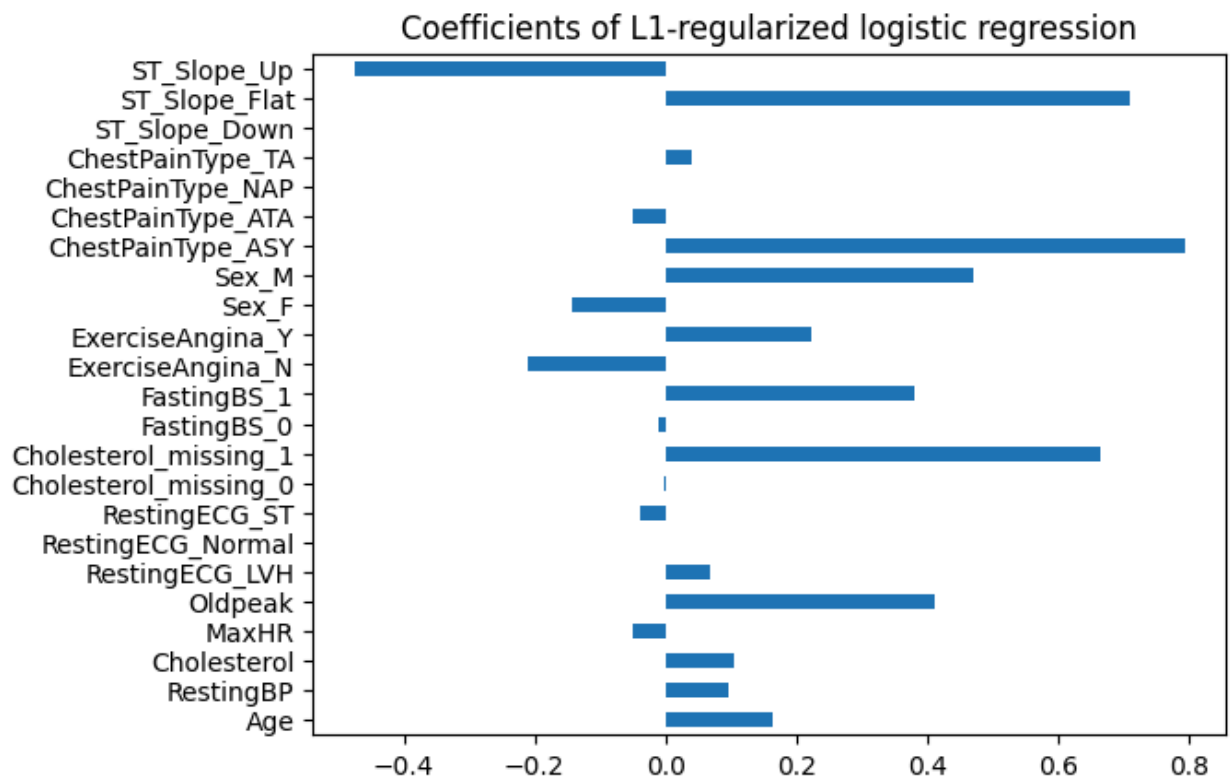
```
In [64]: feature_names
```

```
Out[64]: array(['Age', 'RestingBP', 'Cholesterol', 'MaxHR', 'Oldpeak',  
               'RestingECG_LVH', 'RestingECG_Normal', 'RestingECG_ST',  
               'Cholesterol_missing_0', 'Cholesterol_missing_1', 'FastingBS_0',  
               'FastingBS_1', 'ExerciseAngina_N', 'ExerciseAngina_Y', 'Sex_F',  
               'Sex_M', 'ChestPainType_ASY', 'ChestPainType_ATA',  
               'ChestPainType_NAP', 'ChestPainType_TA', 'ST_Slope_Down',  
               'ST_Slope_Flat', 'ST_Slope_Up'], dtype=object)
```

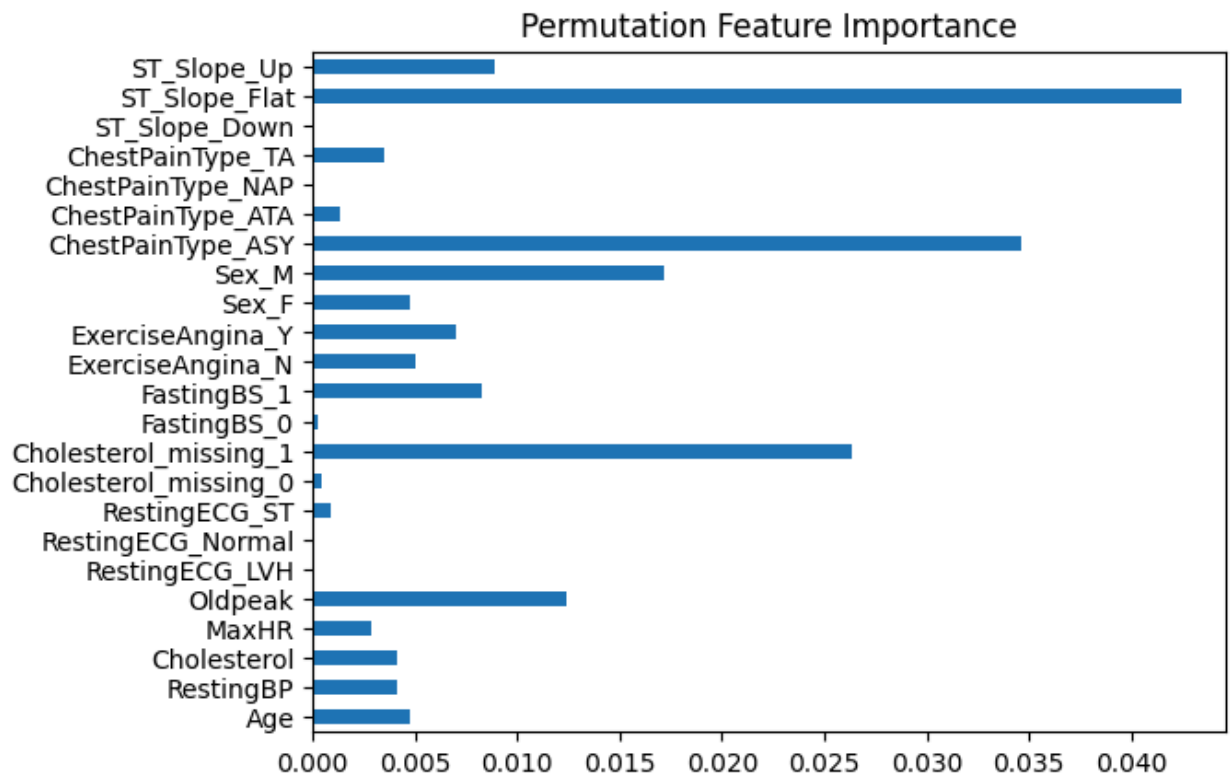
```
In [65]: [np.exp(x)/(1 + np.exp(x)) for x in log.coef_[0]]  
# log.coef_
```

```
Out[65]: [np.float64(0.5411985259836914),  
          np.float64(0.5241257202809901),  
          np.float64(0.5262590982312434),  
          np.float64(0.4877516245661975),  
          np.float64(0.601591179089925),  
          np.float64(0.5170911370929667),  
          np.float64(0.5),  
          np.float64(0.49037495754790394),  
          np.float64(0.49922008222070346),  
          np.float64(0.6604431799094693),  
          np.float64(0.49776596659989186),  
          np.float64(0.5942224847438357),  
          np.float64(0.4479958585035812),  
          np.float64(0.5559405528913449),  
          np.float64(0.46473851719782816),  
          np.float64(0.6154556565994432),  
          np.float64(0.6886415183455356),  
          np.float64(0.48750366900183467),  
          np.float64(0.5),  
          np.float64(0.5104146149099057),  
          np.float64(0.5),  
          np.float64(0.6707508621494465),  
          np.float64(0.38324746526482956)]
```

```
In [66]: pd.Series(log.coef_[0], index=feature_names, name="importances")\  
         .plot.barh(title="Coefficients of L1-regularized logistic regression")  
plt.show()
```



```
In [67]: from sklearn.inspection import permutation_importance
pd.Series(permutation_importance(log, X, y)["importances_mean"], index=feature
          .plot.barh(title="Permutation Feature Importance")
          plt.show())
```



Consider the following setting: A researcher is interested in the important variables and their influence on the label. They have fitted the Logistic Lasso Regression to determine the important variables. Then, they train a Logistic Regression solely on these variables and use this model to make conclusions.

Elaborate why this would be a good or bad idea (1 Pt).

- Bad idea: It is not guaranteed that Lasso retrieves the true relevant feature set. We only learn associations, not causal relationships. If we omit the features with low coefficient values, we might omit variables relevant for causal interpretation.
- Good idea: more interpretability as we have less features now

Q3: Multi-Layer Perceptrons (5 Pts)

Implement a simple MLP, train it on the dataset, and report test set performance (2 Pt).

```
In [68]: from sklearn.neural_network import MLPClassifier
mlp = MLPClassifier(
    hidden_layer_sizes=(128,64), activation='relu', solver='adam', alpha=0.0001,
    batch_size='auto', learning_rate='constant', learning_rate_init=0.001, power_t=0.5,
    max_iter=1000, shuffle=True, random_state=None, tol=0.0001, verbose=False,
    momentum=0.9, nesterovs_momentum=True, early_stopping=False, validation_fraction=0.1,
    epsilon=1e-08, n_iter_no_change=10, max_fun=15000)

mlp.fit(X,y)
```

Out[68]:

MLPClassifier		
Parameters		
	hidden_layer_sizes	(128, ...)
	activation	'relu'
	solver	'adam'
	alpha	0.0001
	batch_size	'auto'
	learning_rate	'constant'
	learning_rate_init	0.001
	power_t	0.5
	max_iter	1000
	shuffle	True
	random_state	None
	tol	0.0001
	verbose	False
	warm_start	False
	momentum	0.9
	nesterovs_momentum	True
	early_stopping	False
	validation_fraction	0.1
	beta_1	0.9
	beta_2	0.999
	epsilon	1e-08
	n_iter_no_change	10
	max_fun	15000

```
In [69]: # make predictions on test set
y_test_mlp_pred = mlp.predict(X_test)
```

```
In [71]: # F1 score: harmonic mean of the precision and recall.
from sklearn.metrics import f1_score
```

```
f1_score(y_true=y_test, y_pred=y_test_mlp_pred, average='macro')
```

Out[71]: 0.202543810062182

```
In [72]: # Balanced accuracy score: average of recall obtained on each class.
from sklearn.metrics import balanced_accuracy_score
balanced_accuracy_score(y_true=y_test, y_pred=y_test_mlp_pred)
```

Out[72]: 0.23015384615384615

Then, visualize SHAP explanations of the outputs of two positive and negative samples and feature importances of the overall model (2 Pt).

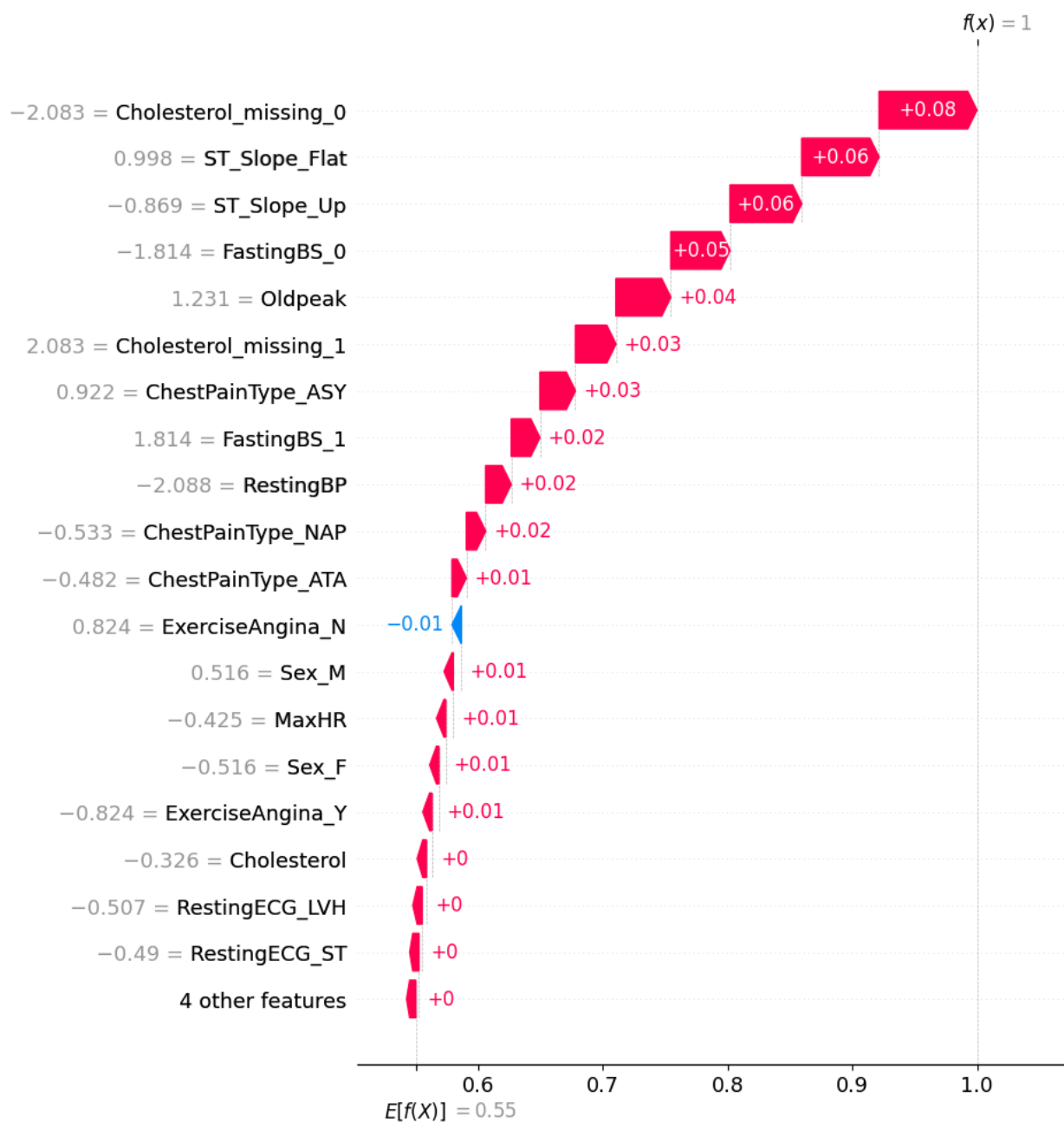
```
In [73]: import shap
```

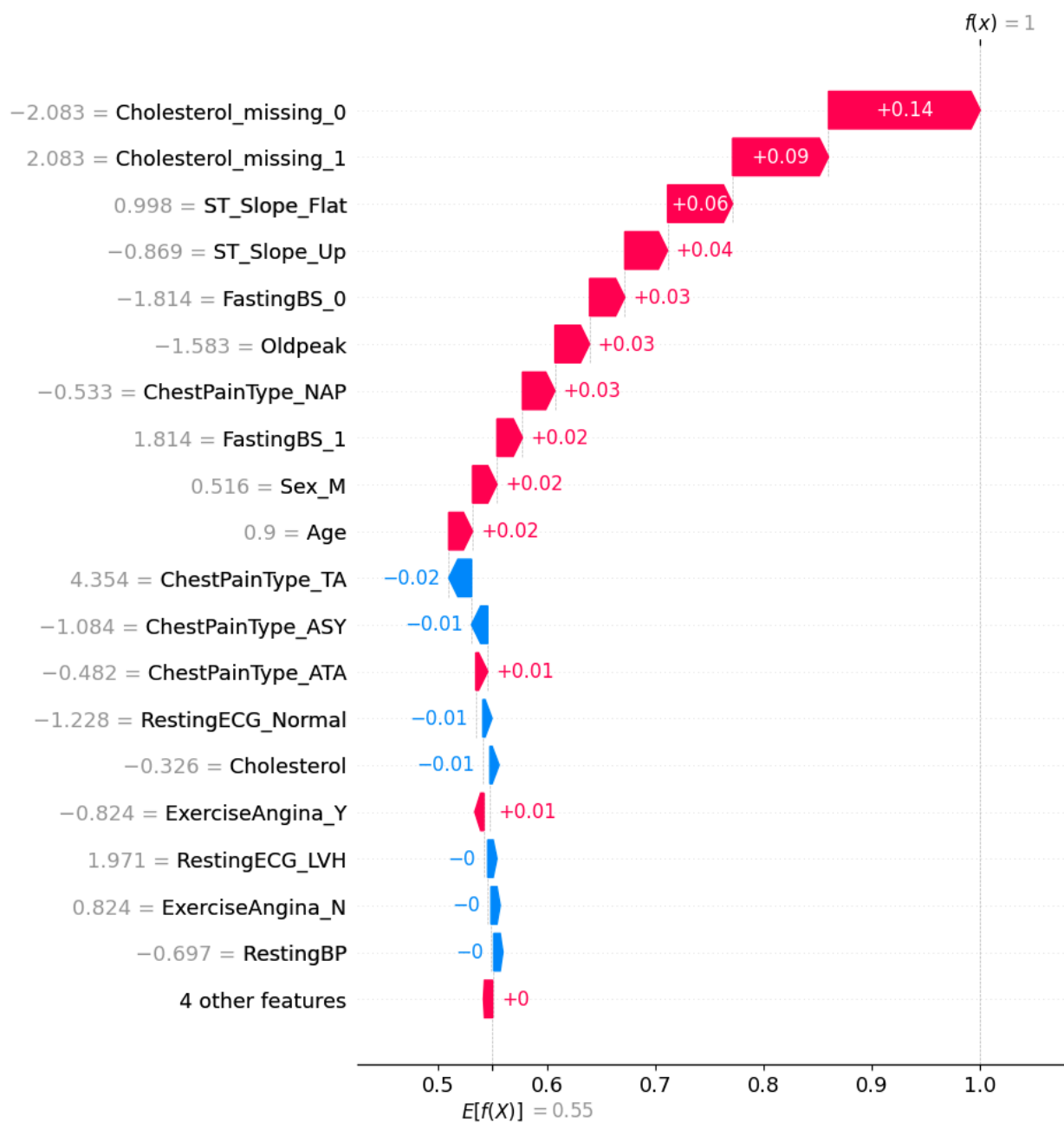
```
In [74]: explainer = shap.Explainer(mlp.predict, X, feature_names=feature_names)
shap_values = explainer(X)
```

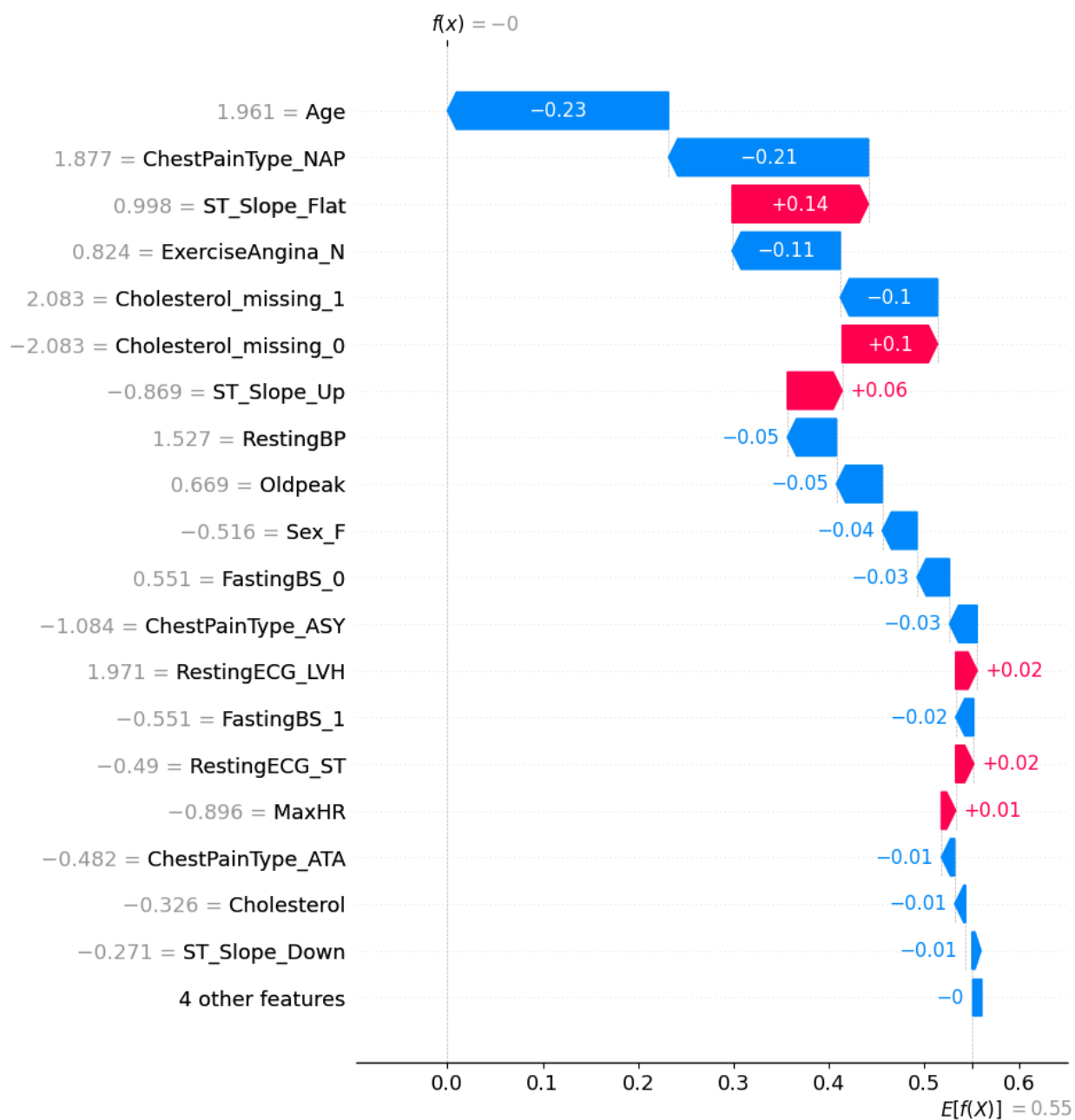
PermutationExplainer explainer: 919it [00:35, 22.83it/s]

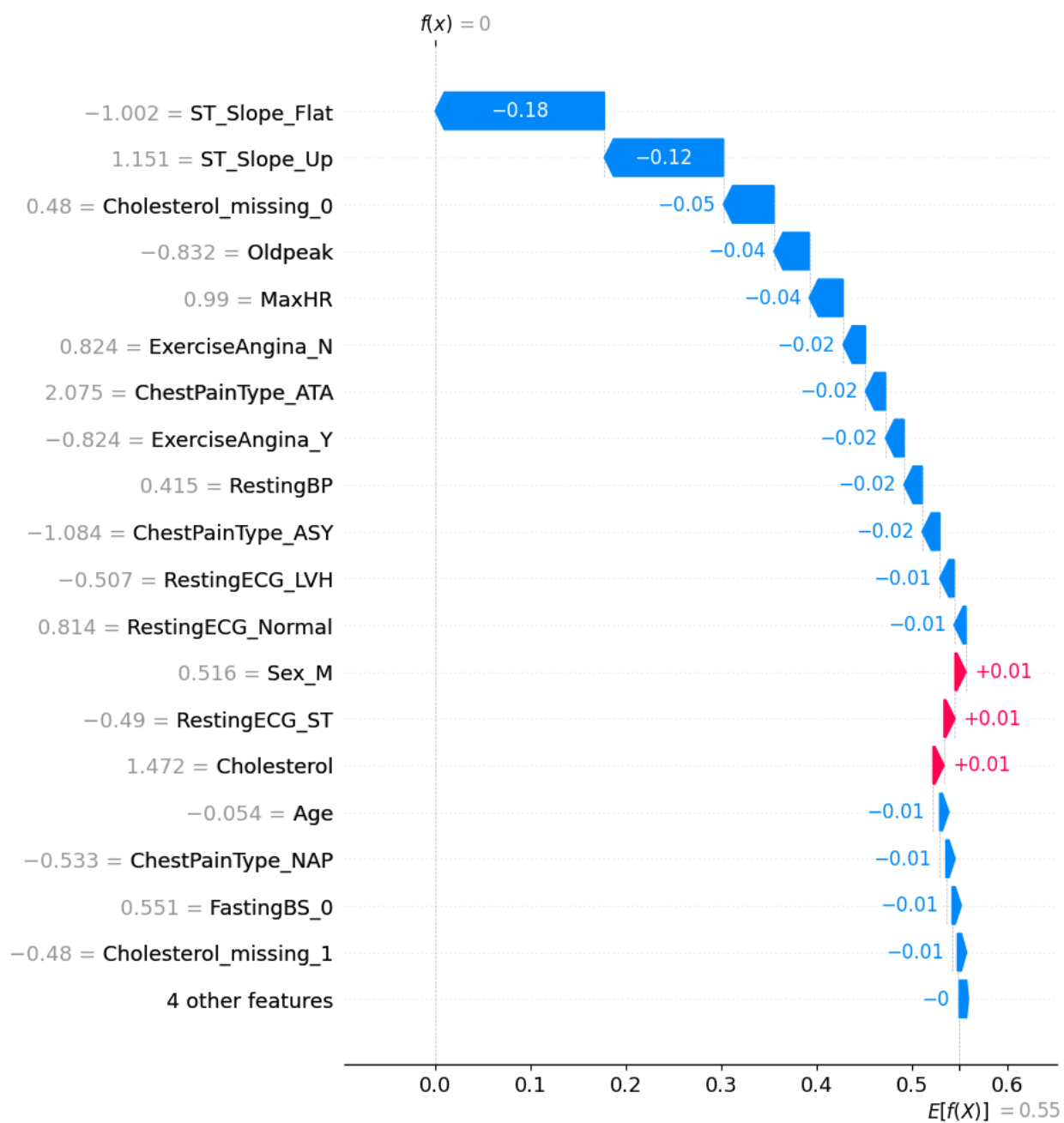
```
In [75]: # get two positive and negative samples
index_negatives = np.random.choice(y[y == 0].index, 2)
index_positives = np.random.choice(y[y == 1].index, 2)
```

```
In [76]: for ix in list(index_positives) + list(index_negatives):
    shap.plots.waterfall(shap_values[ix], max_display=20)
plt.tight_layout()
plt.show()
```

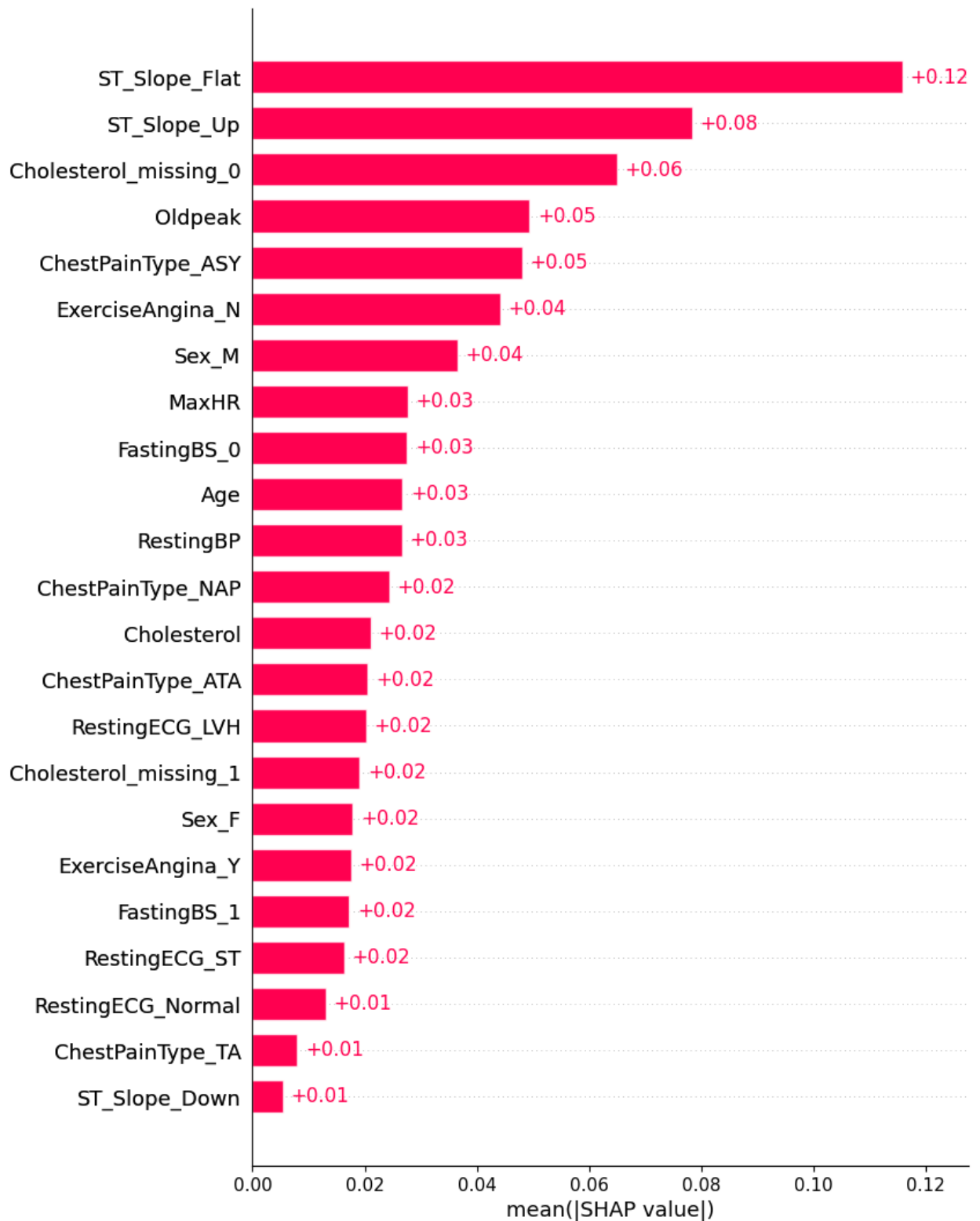






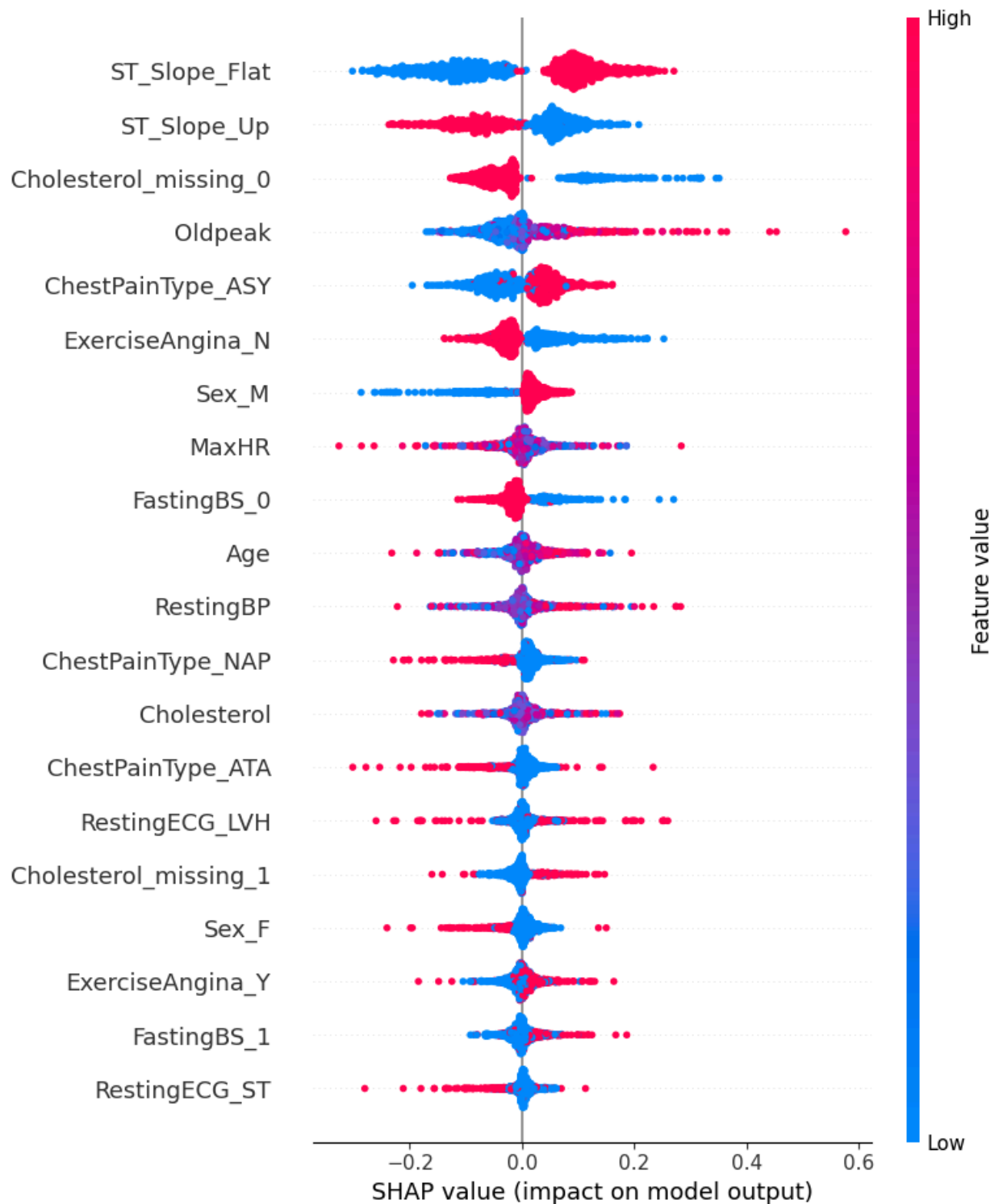
<Figure size 640x480 with 0 Axes>

```
In [77]: # feaature importances of the overall model = mean shapley value over all data
shap.plots.bar(shap_values, max_display=None)
```



```
In [78]: # SHAP values over all data
shap.summary_plot(shap_values)
```

```
C:\Users\gyanr\AppData\Local\Temp\ipykernel_25052\174155317.py:2: FutureWarning: The NumPy global RNG was seeded by calling `np.random.seed`. In a future version this function will no longer use the global RNG. Pass `rng` explicitly to opt-in to the new behaviour and silence this warning.  
shap.summary_plot(shap_values)
```



Are feature importances consistent across different predictions and compared to overall importance values (1 Pt)?

We look at three most important feature for individual explanations and overall feature importance:

- the order and set of the three most important features is not consistent across the 4 predictions and also not consistent with the overall feature importance
 - e.g. Sex_M, ChestPainType_NAP, ST_Slope_Up appear only once
 - e.g. Oldpeak is second most important feature and also appears only once
- however, there is significant overlap; e.g. ST_Slope_Flat is the most important feature and appears for all predictions

Negative sample 1:

- ChestPainType_ATA
- ST_Slope_Flat
- Cholesterol_missing

Negative sample 2:

- ST_Slope_Flat
- ChestPainType_NAP
- Oldpeak

Positive sample 1:

- Cholesterol_missing
- ST_Slope_Flat
- ST_Slope_Up

Positive sample 2:

- ST_Slope_Flat
- Sex_M
- ChestPainType_ATA

Overall feature importance (mean shapley value across all data)

- ST_Slope_Flat
- Oldpeak
- Cholesterol_missing

Conceptually, how does the model compare to Logistic Regression and MLPs (1 Pt)?

A generalized additive model (GAM) is of the form $g(E[y]) = \beta + f_1(x_1) + f_2(x_2) + \dots + f_k(x_k)$.

- Logistic regression is a generalized linear model with the logit function $g(p) = \log(p/(1-p))$ as link. It is a GAM with f_i restricted to be linear.
- Multi layer perceptrons (MLP) are feed-forward neural networks that share their weights among all the input features.
- Neural additive models (NAMs) are linear combinations of neural networks f_i that attend to single input features.

NAM vs. logistic regression

- NAMs and logistic regression are GAMs and thus both have separate weights for separate features; NAMs parametrize the functions f_i with non-linear neural networks and are thus able to fit arbitrarily complex relationships between the feature and the target; logistic regression in contrast is a GAM with f_i restricted to be linear

NAM vs. MLP

- NAMs have different networks for different features whereas MLPs process all input features at once and share their weights among the input features.
- NAMs are GAMs, MLP are not.

In contrast to MLPs and logistic regression, NAMs can be extended to multitask learning by allowing multiple subnets per input feature.

In []: